

AI4SE

人工智能时代的软件工程 思想与实践

从认知重塑到能力落地，构建AI时代的软件工程新范式

课程目录



01 认知重塑

重新理解AI与人的关系
建立正确的AI4SE认知框架



02 工程哲学

重新审视软件工程的本质与方法
银弹论、工程化、代码质量三层模型



03 需求与设计

从模糊想法到精确规格的转化
四大核心原则与Spec先行



04 编码

人机协作编码的最佳实践与方法论
小步快跑、结对调试、软件复用



05 测试与质量保障

构建AI编码的质量护城河
五维度分类、后置评审、测试策略



06 过程体系

从敏捷到CMMI，从OSSP到Harness
HMM成熟度模型与LLM赋能

认知重塑

重新理解AI与人的关系，建立正确的AI4SE认知框架

AI编程模式核心感悟 · 两类工作哲学 · AI能做而人做不到的五件事

封闭式工作 → AI主场

特征：规则明确、结果可验证、存在有限解

AI表现：如鱼得水

代码生成 · 测试编写 · 文档同步 · 格式转换

数据清洗 · 正则匹配 · API调用 · CRUD操作

AI负责处理确定性

开放式工作 → 人类主场

特征：没有标准答案、结果是无限解、依赖主观判断

AI表现：只是灵感催化剂

架构设计 · 需求定义 · 战略决策 · 团队管理

创新突破 · 利益权衡 · 价值判断 · 人际协作

人类负责探索不确定性

核心洞察：人类的终极价值在于定义规则、突破规则、在模糊中寻找方向

文档与代码同步

人类痛点：写完代码不写文档，
文档与代码永远不同步

AI优势：每次代码变更后自动同步
文档，无需人类意志力

理想主义照进现实

需求追踪矩阵

人类痛点：需求→设计→代码→
测试
完整追溯几乎不可能

AI优势：不知疲倦地维护
全链路双向追溯矩阵

工程纪律的革命

设计先行

人类痛点："先动手再说"的冲动，
跳过设计直接写代码

AI优势：不抗拒先做设计再写代
码
严格执行Spec先行流程

规范化的守护者

核心洞察：AI没有"人性弱点"——不知疲倦、不拖延、不妥协、不走捷径

测试驱动开发（TDD）

人类感受：先写测试是"延迟满足"，本能抗拒

AI真相：

AI没有"满足延迟"的心理感受

红→绿→重构循环，AI执行得天经地义

人类抗拒的"负担"变成AI的"默认模式"

质量门禁

人类感受：会被deadline压垮而跳过检查

AI真相：

严格坚守每一项质量检查，不妥协不跳过

覆盖率检查→坏味道扫描→安全审计→回归验证
所有门禁零遗漏，不会因为"赶时间"而妥协

AI4SE的终极价值

不是让人类写代码更快，而是让人类更守纪律，且让守纪律不再痛苦

🎯 What与How的严格区分

人从"写How"上升到"定义What"

- 算法、数据结构、设计模式→AI
- 业务目标、规则、分支→人
- 人审核AI选的How是否合理

从"代码实现者"到"需求定义者"

🧠 AI = "聪明的傻子"

"聪明": 快速生成正确代码

"傻子": 不追问、不质疑、不补充

正是因为AI是"傻子", 你才被迫把需求想清楚、写清楚

倒逼开发者提升思维严谨性

💬 沟通重点变了

从"写给机器执行"→"写给AI理解"

编程范式转变:

命令式编程 → 意图式编程

GIGO原则被放大十倍

指令质量 = 输出质量

核心洞察：编程范式从"命令式"转向"意图式", 人从"翻译官"变为"指挥官"

角色跃升路径

代码实现者



需求定义者



AI编排师



业务架构师

✅ 验证对象变了：从代码逻辑 → 业务逻辑

代码逻辑的正确性由AI保证（语法、类型、空指针等）

业务逻辑的正确性由人保证（规则对不对、分支全不全、边界有没有）

责任上移：从"技术债"到"业务债"

🔄 完备性左移：以前不愿意做的，现在必须早做

规则不全→AI瞎猜 | 边界没想→AI瞎填 | 异常没写→AI瞎处理

AI逼你前置思考，走向"规格驱动开发"

软件工程的理想状态，AI让它成为现实

🤖 人机关系变了：从"人用工具"到"人指挥AI用工具"

以前：人是操作者，需要懂工具怎么用

现在：人是指挥官，需要懂任务怎么拆解、什么结果算好、什么算异常

从"操作技能"转向"判断力和指挥力"

⚠️ 错误类型变了：从语法/逻辑错误 → 意图偏差

传统：空指针、死循环、数组越界 → AI时代几乎消失

新错误：语法正确、逻辑自洽，但完全不是你想要的业务

"意图偏差"比语法错误更难发现、更难调试

传统模式

先写代码，后补文档

先上线，后补异常处理

VS

AI模式

规格先行，代码由AI从规格生成

异常和边界必须前置定义

测试定位变了

从"验证代码有没有写对"
→"验证规格有没有被正确实现"
+ "规格本身有没有遗漏"
测试用例从规格自动生成
人的工作→审核测试覆盖
从手写测试到审核覆盖

文档价值变了

以前："写给以后的人看"→ 常常被忽略
现在："给AI吃的"→ 代码的唯一来源
没有规格→无法生成正确代码
从"副产品"到"一等公民"

能力模型变了

以前：精通语言、框架、算法
现在：把模糊需求→精确规格
识别业务分支和边界
判断AI输出是否符合意图
逻辑严谨性 > 技术熟练度

感悟全景：从操作者到指挥官的跃迁

认知重塑



质量前移



能力进化

人的价值从"操作能力"转向"定义能力+判断能力"
AI的"傻子"特性，恰恰强迫人成为更严谨的思考者

我们不是因为AI而放弃思考，而是因为AI而必须更深刻地思考。

Q1 角色转变

从 "程序员"
到 "AI编排师"

不再亲自执行每个技术步骤，而是定义目标、约束与验收标准
将AI编排为可复用的 workflows

Q2 能力转变

从 "技术深度"
到 "业务抽象+逻辑严谨"

AI倒逼人必须想清楚"做什么"，模糊的输入只会得到模糊的输出
业务抽象能力取代了语法记忆能力

Q3 流程转变

从 "先写后改"
到 "先想清楚再动手"

Spec先行、设计先行——想清楚了就不用做两遍
减少无效迭代，追求一次做对

Q4 标准转变

从 "代码行数"
到 "功能点+质量指标"

AI让代码行数彻底失效——有价值的功能交付才是唯一尺度
测试覆盖率+缺陷密度+坏味道密度衡量真实质量

Brooks, 1986 —— "没有银弹"

没有一种技术或管理上的进步能够独立承诺在10年内将软件生产率、可靠性或简洁性提高一个数量级。

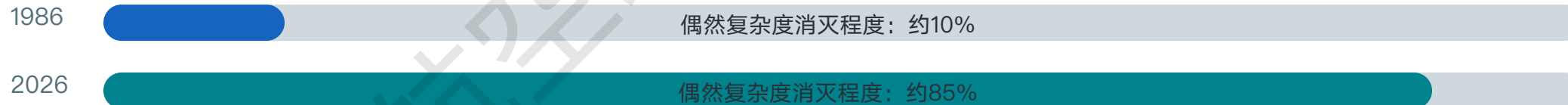
AI时代重新审视

AI没有改变"本质复杂度不可消除"的结论，但将"偶然复杂度"的消灭推到了前所未有的程度

曾经被认为是"本质"的复杂度（如需求理解、设计沟通），在特定场景下正被AI逐步转化为"偶然"

核心启示：银弹论的边界在移动 —— 拥抱AI消灭偶然复杂度，将人的精力集中到真正不可替代的本质复杂度上

银弹论的边界在移动



偶然复杂度 AI可消灭

代码生成 —— 从自然语言到可运行代码

测试编写 —— 自动生成全面测试用例

文档同步 —— 代码变更即文档更新

格式转换 —— 任意数据格式互转

代码翻译 —— 跨语言代码转写

配置管理 —— 自动化环境配置

AI让"体力活"趋近于零
最强辅助子弹

本质复杂度 人类不可回避

模糊业务理解 —— 真实世界需求从来不是非黑即白

架构权衡 —— 性能/成本/可维护性的三角取舍

测试预言 —— "正确的输出是什么"本身就是最难的问题

最终责任承担 —— 代码上线后果只能由人负责

人际协作 —— 跨团队沟通、利益协调、共识达成

"脑力活"的重要性反而被放大
永远不可替代的领域

底线：AI能让软件工程的"体力活"趋近于零，但"脑力活"的重要性反而被放大

四层错误剖析

第一层：语法错误

类型不匹配、语法冲突
→ 编译器可消除

第二层：逻辑错误

条件判断、状态机、计算顺序
→ 需人类审查

第三层：边界工程错误

null/空串/超大/并发/超时
→ 需测试闭环

第四层：安全业务错误

权限、合规、业务规则
→ 需领域知识

核心结论

AI本质上是概率性文本接龙，而非逻辑推导

GPT不会"推理"，它只是在统计频率最高的下一个token

但 —— 人机协作可达到零缺陷

"人定规则 + AI落地 + 测试闭环" 范式

可以产出通过所有测试、可落地的零缺陷代码

关键定位：AI是副驾驶，而非自动驾驶
方向盘始终在人类手中

工程哲学

重新审视软件工程的本质与方法

银弹论新思考 · 工程化必然性 · 代码质量金字塔 · 去模糊化方法论

痛点1: 忘性大

对话上下文丢失
早期约定被遗忘

第1轮: 约定用A方案
第20轮: AI已忘记A方案
结果: 代码前后不一致

→ 对策

规则外置: 写入Skill/规则文件
契约文档化: 显式合同
回归测试护城河

痛点2: 自以为是

遇到模糊信息时
基于概率"脑补"而非询问

人: "优化性能"
AI脑补: 加缓存 (未必对)
结果: 答非所问

→ 对策

去模糊化沟通
精确输入: 量化+样例+边界
Sample-First 先样例

痛点3: 知识陈旧

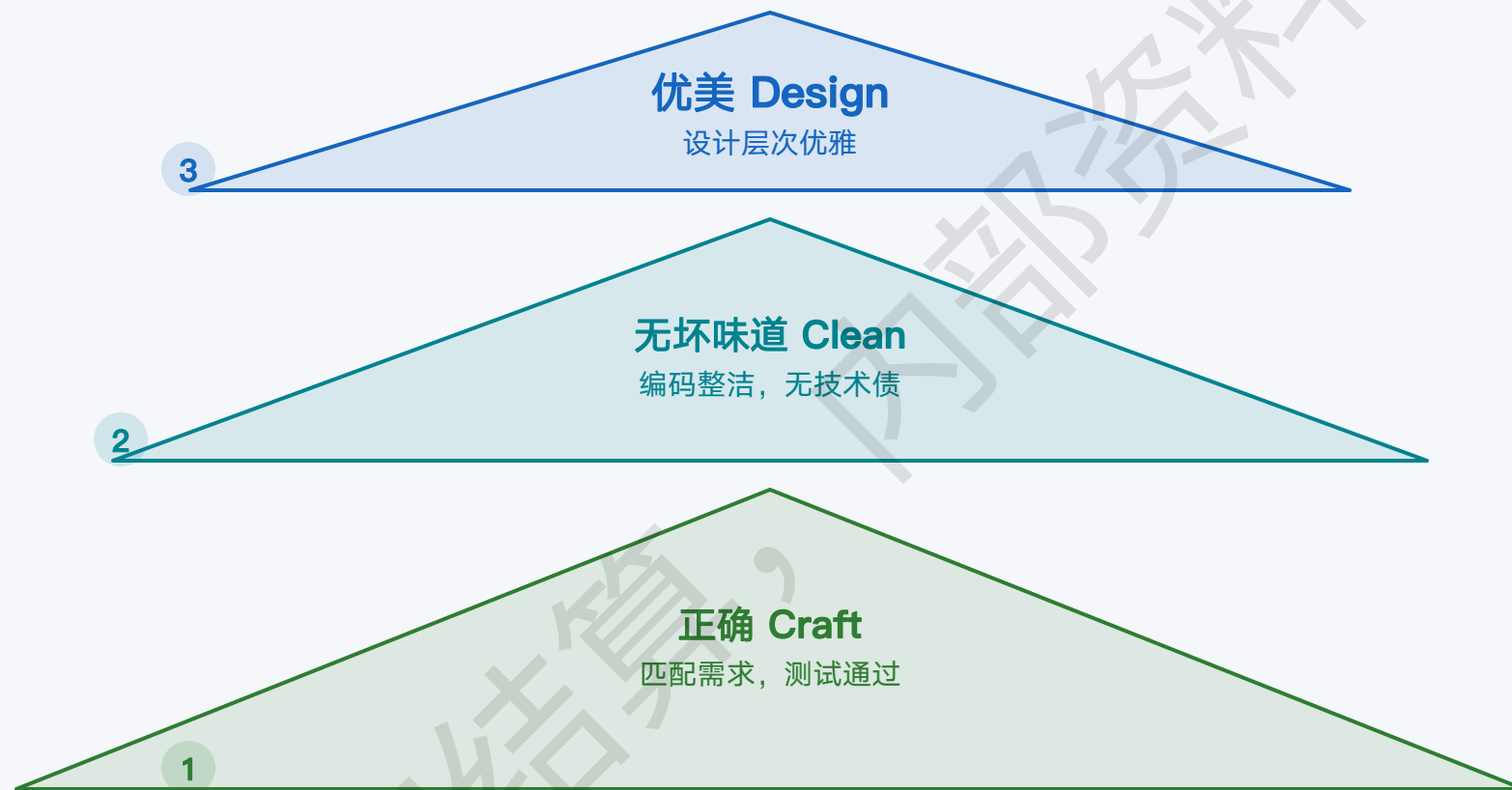
训练数据截止
导致使用旧API/旧框架

要求: 用React 19
AI输出: React 18 API
结果: 不兼容、报错

→ 对策

RAG知识库注入最新知识
Skill注入最新API文档
代码审查机制

核心洞察: 三大痛点是可管理的 —— 不需要等AI变聪明, 只需学会驾驭它



底层：正确（Craft） 匹配需求，靠TDD约束 先写测试 → AI生成代码 → 测试通过 = 正确

中层：无坏味道（Clean） 编码整洁，靠坏味道自动扫描与重构 检测圈复杂度、重复代码、长方法

顶层：优美（Design）—— 设计层次优雅，靠意图识别、模式推荐和SOLID自检。从代码堆砌到架构之美

3 设计决策（优美层）

确定架构模式、组件边界、接口契约
选择设计模式、定义SOLID原则落地方案
人类主导 —— AI辅助分析选项



2 TDD实现（正确层）

红 → 绿 → 重构循环，每次只改一个测试
先写失败测试 → AI生成最小实现 → 重构优化
AI执行 —— 人类审查测试逻辑



1 坏味道扫描（无坏味道层）

自动检测圈复杂度、重复代码、长方法、耦合度
自动建议重构方案，AI执行重构，回归测试验证
AI自动执行 —— 自动化程度最高

金字塔从难到易：优美层最考验判断力 → 正确层靠方法约束 → 无坏味道层自动化程度最高

01

传统工程化的天花板是人

人会累 —— 持续注意力有限
人会妥协 —— deadline压力下跳过流程

人会抵触 —— 繁琐流程让人抗拒
人会走捷径 —— 测试跳过、文档不写

这不是技术问题，
是人性问题

02

AI放大了不规范的成本

不规范 = AI输出不可控
返工成本成倍增加

模糊需求 → 跑偏代码 → 多次返工
小洞不补 → AI制造大窟窿

AI时代不规范的成本
倒逼工程化

03

AI完美匹配工程化需求

永不疲惫的执行者
持续注意力，不疲劳不偷懒

无捷径倾向 —— 不走歪路
硬性约束遵守 —— 不妥协流程

"严格执行工程化"
首次成为可能

核心洞察：AI不仅需要工程化护航，而且AI本身就是工程化的最佳执行者

第三层

Skills

可复用的AI workflow 模板

封装最佳实践，将经验转化为可执行的结构化指令集

第二层

Standards

编码规范、命名约定、架构约束

将团队共识固化为AI可读取的标准化规则

第一层

Tools

自动化测试、代码扫描、CI/CD流水线

不可跳过的自动化质量门禁，让AI输出有据可查

基座

Principles

不可妥协的工程底线

每次变更必须通过全部测试 —— 这是铁律

四层架构自底向上：原则为基 → 规范为网 → Skill为桥 → 工具链为盾 —— 缺一不可

四大核心原则将AI开发从试错式编码转变为规范化工程落地



完备规划

先定规则再写代码

需求文档→业务事件→
业务能力→结构化描述
→ workflow编排



分级MVP

嵌套拆解为MTF

大需求→中需求→
小需求→
最小可验证单元



增量实现

小步叠加、只增不改

每次只添加一个
已验证的最小单元
——不破坏已有稳定



局部修改

最小影响域

改哪里说哪里，
圈外不动
——杜绝全局重构



核心逻辑：四个原则环环相扣 —— 规划定义边界 → MVP确定粒度 → 增量控制节奏 → 局部确保安全

"鼓励有效增量，减少无效迭代"是四大原则的核心理念

✓ 有效增量

每个增量都是可验证、可交付、不破坏已有功能的独立单元

特性一：每个增量可以独立部署和回滚

特性二：增量通过时累积已有测试全部回归

特性三：每个增量对应一个可演示的业务价值

特性四：增量之间按依赖顺序线性叠加，不交叉

⚠ 无效迭代

大需求一次扔给AI → 多轮修改返工 → 质量不可控 → 交付延期

症状一：上下文窗口被撑爆，AI遗忘早期约定

症状二：每次修改都可能破坏已有功能

症状三：缺乏可验证的中间节点

症状四：调试成本随规模指数级增长

核心公式

有效交付 = 完备规划 × 分级MVP × 增量实现 × 局部修改

💡 四个原则是乘数关系 —— 任何一个为零，整体交付都无法成功。不是"或"，是"且"。

核心主题

AI时代的工程哲学：拥抱AI效率，坚守工程纪律

01

哲学一：AI不是银弹，但能让偶然复杂度趋近于零

AI消灭体力活，让人聚焦本质复杂度 —— Brooks论断成立，但边界在移动

02

哲学二：AI倒逼工程化

不规范的成本被AI放大，守纪律的价值被AI彰显 —— 工程化从"应该"变成"必须"

03

哲学三：代码质量三层模型

正确 → 无坏味道 → 优美，逐层递进。三层缺一不可。

04

哲学四：小步快跑，增量迭代

完备规划，最小可测特性，增量实现，局部修改

关键行动：建立工程化体系，用去模糊化沟通，让AI在约束中发挥最大效能

需求与设计

从模糊想法到精确规格的转化之道

需求是AI编程的第一道关卡，设计决定了代码质量的上限

AI无法主动追问模糊需求，开发人员必须把需求转化为精确的结构化规格



AI可读规格 5项检查单



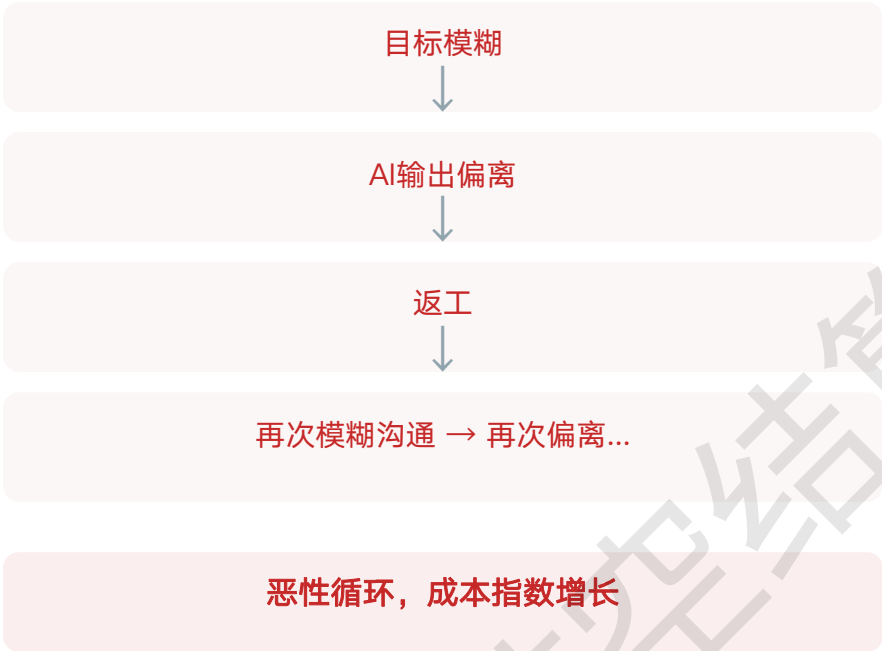
核心原则

你给AI的指令越清晰，AI给你的回报越精准 —— 精确度与输入清晰度严格正相关

Garbage In → Garbage Out

1:10定律 —— 前期投入1分钟去模糊化，可节省10分钟返工时间

模糊沟通的代价



7条去模糊化方法论

- 1. 验收标准可测：给定输入→期望输出可验证
- 2. 量化任务目标：响应时间500ms→200ms
- 3. 量化篇幅规范：800–1000字接口设计文档
- 4. 量化输出格式：{code, data, message}结构
- 5. 明确内容边界：包含什么、不包含什么
- 6. 清晰应用场景：上下文环境明确
- 7. 精确逻辑颗粒度：功能边界与依赖明确

核心原则：摒弃所有形容词副词沟通，实现确定性交付

将模糊业务描述转化为可执行的结构化需求

BEFORE 模糊需求

"做一个用户管理系统"

AI的反应:

"好的, 我来做一个用户管理系统..."

- 生成一个简单的CRUD页面
- 没有权限管理、没有角色区分
- 需要多轮返工

问题诊断:

- 目标模糊: 是内部系统还是对外产品?
- 角色缺失: 有哪些用户角色?
- 功能域未定义: 包含哪些功能?
- 非功能需求缺失: 并发、性能、安全?
- 全局: AI无从下手, 只能猜测

AFTER 去模糊化后

目标: 实现企业内部用户的生命周期管理

角色: 普通用户、部门管理员、系统管理员

功能域: 注册登录、权限管理、
部门归属、状态流转 (启用/禁用/离职)

非功能: 1000并发、响应<200ms、加密存储

转变效果:

- 从一句话描述到AI可直接生成代码的结构化规格
- AI能精确理解需要实现哪些功能模块
- 角色和边界明确, 生成的代码有明确的目标受众
- 非功能需求明确, 代码从一开始就考虑性能和安全

一句话描述 → 结构化规格 → AI可直接生成代码 = 1:10 效率提升

五步转化法：需求文档 → 业务事件 → 业务能力 → 结构化描述 → workflow编排

Step 1 需求文档

收集并整理原始需求，去模糊化 —— 把"做一个用户系统"转化为明确的功能域和优先级

Step 2 业务事件

识别关键业务事件（用户注册、订单创建、角色变更） —— 每个事件是一个独立可交付单元

Step 3 业务能力

映射每个事件需要的业务能力 —— 用户CRUD、权限校验、状态流转、审计日志

Step 4 结构化描述

用统一模板描述每个能力：输入/输出/边界/异常 —— 让AI无歧义理解每一个原子任务

Step 5 workflow编排

将能力串联为完整 workflow，形成AI可执行的开发任务序列 —— 从需求到代码的最后一步

需求文档 → 业务事件 → 业务能力 → 结构化描述 → workflow编排 → AI可执行代码

01 完备规划法则

先想清楚再让AI动手
想清楚就不用做两遍

需求阶段的1小时投入
= 编码阶段节省1天

完备规划不是"慢",
而是最高ROI的前期投资

🕒 投入产出比高达 1:8

02 Spec先行法则

Spec是人与AI之间的"合同"
双方以此为唯一真相源

没有完整Spec就不写
一行代码

Spec先行不是"慢起步",
而是让每一步都有据可依

📄 Spec = AI的确定性输入

03 分层拆解法则

从粗粒度到细粒度逐层拆解
每层独立验证

禁止一次把全部需求
扔给AI

系统→子系统→模块→
功能→最小可验证单元

🔗 逐层拆解 = 逐层验证

💡 三条法则层层递进：完备规划定义"做什么" → Spec先行确认"做成什么样" → 分层拆解确保"怎么做对"

往前看：需求探索

- 🔍 识别潜在风险：需求阶段的模糊点和高风险区域
- ✅ 明确验收标准：每个需求的"通过"定义
- 🎯 设计测试策略：基于需求的五维度测试方案
- 🔬 技术预研：不确定技术方案的早期验证

往后看：缺陷复盘

- 🔍 现象捕捉：Bug的完整触发条件和影响范围
- 🔍 根因分析：5 Why法追溯根本原因
- 📖 教训沉淀：将根因转化为可复用的测试规范
- 🛡️ 预防措施：更新测试策略，堵住同类缺陷入口

闭环：往前看（探索） → 测试执行 → 往后看（复盘） → 规范沉淀 → 持续改进

核心观点：只有将探索与复盘统一，将教训沉淀为规范，才能实现持续改进的闭环

将资深需求工程师的方法论注入AI，实现30分钟从模糊想法到完整需求规格

12需求元素体系

角色（Actor）	目标（Goal）	前置条件
后置条件	主流程	异常流程
数据项	校验规则	业务规则
非功能需求	约束条件	验收标准

12元素构成完整的需求元模型
——每个元素精确描述一个维度

三大核心能力

-  **结构化对话引导**
VIST+AED校验，逐步澄清模糊需求
-  **自动完备性检查**
识别遗漏、矛盾、歧义，确保12元素全部覆盖
-  **CLOUD拆分**
将大需求合理拆解为独立可交付单元
-  **效果：大幅降低对专业需求工程师的依赖**

五阶段Vibe Coding方法论 —— 从模糊想法到可运行的软件

P1

💡 想法注入

通过与AI对话逐步注入领域方法论，让AI理解需求分析师的专业知识体系

P2

📄 Spec生成

基于注入的知识生成结构化Spec，作为开发的唯一真相源

P3

🔬 技术预研

系统性技术预研，让AI列出选项由人做决策，用实验替代猜测

P4

🧩 构件提取

从对话中提取11组复用构件（提示词模板、数据结构、校验规则等）

P5

🔍 模式识别

提取6种设计模式（双面板布局、展示-逻辑分离、事件驱动等），形成可复用架构资产

"增量注入知识、让AI列出选项你做决策、Spec先行再动代码、实验替代猜测"

★ 关键原则

🔄 增量注入

不要一次把所有知识给AI，分阶段注入

第一阶段：领域概念 → 第二阶段：方法论 → 第三阶段：具体约束

👉 人做决策

AI列出选项+优劣分析，人做最终选择

技术栈选型、架构决策、设计取舍 —— 永远由人拍板

📄 Spec先行

先完成完整Spec，再开始写一行代码

Spec是人与AI之间的"合同"，双方以此为唯一真相源

⚠️ 避坑指南

坑1 跳过技术预研

不要在技术栈未确认时直接写代码

用实验替代猜测：让AI写小demo验证关键假设

坑2 Spec未完成先编码

不要在Spec未完成时开始编码

Spec不完整=返工隐患。宁可多花1小时写Spec

坑3 让AI做架构决策

不要让AI替你做出重要的架构决策

AI善于列出选项、善于执行，但不善于权衡取舍



核心心法：Vibe Coding的本质是通过结构化对话将人的认知逐步"迁移"到AI中 —— 每次对话都是一次知识蒸馏

第三章 需求与设计

核心认知

AI无法主动追问模糊需求 —— 模糊的输入必然产生模糊的输出

方法论

四大核心原则 + 五步转化法 + Spec先行

工具支撑

AI需求分析师、结构化需求模板、去模糊化检查单

★ 关键行动：把需求工程能力提升为团队的核心竞争力

编码

人机协作编码的最佳实践与方法论

AI编码不是"让AI写代码", 而是"人与AI高效协作编码"

AI天生"喜新厌旧" —— 面对全新需求表现极好，一旦涉及历史代码就极易出错

AI表现曲线：随依赖复杂度递减

🚀 全新独立需求 → 表现优秀

↘️ 有少量依赖 → 表现下降

⚠️ 多模块接口 → 易出错

历史遗留代码 → 越改越乱

三大对策

🔗 1. 契约先行

在开始编码前定义清晰的接口契约
输入/输出/异常条件 —— 显式化所有隐性约束

🛡️ 2. 回归测试护城河

每次变更都跑完整回归测试
自动化的安全网，让AI的"健忘"无处遁形

📦 3. 原子化提交

每次只提交一个独立的功能变更
变更越小，AI"翻车"的影响面越小

最优策略：不是"先随便写再慢慢改"，而是力争一次成功

把复杂需求拆解为小到可验证的任务 —— 人类定流程、拆任务，AI做执行、写代码

⚠️ 为什么大需求直接给AI注定失败

原因1 AI是语言模型而非逻辑引擎 —— 无法自主规划复杂任务的执行顺序

原因2 AI缺乏真实执行环境 —— 无法通过运行代码来验证自己的输出

原因3 上下文窗口限制 —— 超过窗口的内容会被遗忘

原因4 训练数据"平均化"陷阱 —— AI倾向给出最常见而非最合适的方案

✅ 正确方法：小步快跑

复杂需求

▼ 拆解为独立任务

任务1 → 执行 → 测试验证 ✓ → 提交

任务2 → 执行 → 测试验证 ✓ → 提交

任务N → 执行 → 测试验证 ✓ → 提交

累积完整系统

💡 核心原则：每个任务确认后才继续下一步 —— 不是"等一下再改"，而是"对了再往下走"

AI调试的默认行为是"看到现象 → 给出最熟悉的解"而非"定位根因"

✓ 9大防坑原则

1. 根因验证：AI给修复前，先确认它找到了真正的根因
2. 权宜之计识别：警惕AI为了快速修复而牺牲架构完整性
3. 初心保持：排查时不被B/C问题分散注意力而忘记A
4. 全链路留痕：每次LLM调用记录提示词和输出
5. 全域自查：修完一个位置后主动检查类似位置
6. 最小修复法：每次只改一处，立即验证，不要批量修复
7. 回归测试验证：每次修复后必须跑完整回归
8. 上下文隔离：不要用同一个对话窗口调试多个Bug
9. 知识沉淀：修复完成后把教训写入Skill和测试规范

🔄 6步标准调试流程

- 🔍 Step 1: 复现 稳定复现Bug，记录最少复现步骤
- 📄 Step 2: 日志收集 收集错误日志、堆栈跟踪、输入数据
- 💡 Step 3: 假设生成 人列可能原因，AI提供分析和补充
- 🔍 Step 4: 根因定位 用最小实验验证每个假设，定位到精确代码行
- 🔧 Step 5: 修复 人确认修复方案，AI执行，只改最少代码
- ✅ Step 6: 回归验证 全量回归测试+全域自查+添加新测试用例

应对隐性契约与硬编码 —— 一个Excel自动化程序的6轮调试教训

🚩 陷阱：B27单元格公式 =MID(B27,7,1)

公式取第7个字符判断等级。程序写入"Maturity Level 5"，第7个字符是"t" → 全盘出错

教训1：隐式契约系统

Excel公式、数据验证、宏共同构成不透明的隐式契约系统
多个环节的隐性规则组合在一起，形成难以发现的陷阱

教训2：字符串位置依赖 = 定时炸弹

应使用结构化数据（如JSON字段）而非位置编码
一旦字符串长度或格式变化，依赖位置读取的逻辑全部失效

教训3：AI不会发现隐性契约

AI只看代码层面的接口，无法发现跨系统的隐式依赖
这类隐性契约必须由人识别并显式化，写入Spec和契约文档

解决方案：显式化 + 契约化

将所有隐性约束写入Spec → AI每次生成代码前自动加载
从"第7个字符"到"level字段"，从位置依赖到语义字段



核心启示：隐性契约是最大的工程质量杀手

—— 任何没有写在文档里的规则都是潜在的故障点。AI编码时代，显式化比以往任何时候都重要。

"提取 → 沉淀 → 登记 → 推荐 → 评审"全闭环复用机制

</> 源码级复用（构件调用）

建立可复用组件清单 → 让AI在编码时主动推荐已有组件
→ 每次复用后记录 → 形成组件使用知识图谱

🔍 设计级复用（机制与模式）

提取设计模式 → 沉淀为Skill → 在开发全流程中主动推荐和校验
→ 让模式复用不依赖人的记忆和经验

全闭环复用机制



新发现的复用机会 → 回到"提取"步骤，形成正反馈循环

★ 核心贡献：让复用从"靠人记忆"变成"AI主动推荐" —— 复用率提升3-5倍

重构不是一次性事件，而是持续改进过程

⚠️ 常见代码问题

重复代码

多处相同逻辑，修改需要同步

死代码

永远不执行的逻辑

霰弹式修改

改一处需要同时改N处

异步竞态

并发状态不一致

过度耦合的消息链

a.b().c().d()深层调用

☆ 关键经验

小步快跑

每次只改一点，改完立刻验证

向后兼容

接口契约不可破坏

优先复用

优先使用公共组件

测试保护

重构前必须有测试覆盖

变更蒸馏

每次重构沉淀经验

🎯 核心原则

建立单一真相源

避免数据/逻辑分散

统一命名约定

团队共识，AI强制执行

代码审查清单

标准化检查项

AI辅助执行

AI执行重构，人做决策

持续改进

不追求完美的重构



核心理念：让AI辅助执行重构而非主导重构 —— 人做决策、AI做执行、测试做保护

小步重构五大优势：风险可控、即时反馈、边做边学、不影响正常开发、容易说服团队

BEFORE Flash提示方案

问题现状：

多个页面各自使用Flash提示机制
→ 每个页面有自己的提示代码
→ 提示样式、行为不一致
→ 修改提示逻辑需要改所有页面

痛点：

- 用户体验不一致
- 维护困难，改一处漏百处
- 新增页面时需复制粘贴旧逻辑
- 没有统一的状态管理



AFTER 统一Toast组件

改进后：

所有页面共享一个Toast组件
→ 样式、行为完全一致
→ 修改一处全局生效
→ 新增页面直接引用即可

效果：

- 用户体验完全一致
- 单点维护，改一处改全部
- 新页面零复制粘贴
- 统一状态管理和自动消失

🔄 过程：每次迁移一个页面 → 立即测试 → 确认无问题 → 迁移下一个

核心原则：重构不是革命而是演化 —— 每次只改一点，立即测试验证，随时可以停下

不只是修复错误，而是管理每一次变化 —— 让AI从"代码生成器"升级为"学习型协作者"

第一层

变更归零

技术归零 + 管理归零

彻底理解每次变更的根因 —— 不是"修好了就行"，而是"为什么会出现这个Bug"



第二层

四环节改进

在需求 / 设计 / 编码 / 测试四个环节找到改进点

Bug不止是编码问题 —— 需求不清、设计缺陷、测试遗漏都是根因



第三层

模式提炼

从具体变更中抽象出可复用的模式和反模式

从"这个Bug是因为字符串位置依赖"到"位置依赖是一种反模式"



第四层

原则蒸馏

将模式上升为原则，注入Skill和规则体系

成果：AI能基于历史经验主动提醒风险、匹配解决方案

第四章 编码

喜新厌旧规律

AI对全新需求表现好，对历史代码需额外谨慎

小步快跑

复杂需求拆解为可验证的最小单元，逐块累积

调试防坑 + 可进化系统

AI默认行为是"给最熟悉解"而非"找根因"，需建立纠偏机制
每次变更不止修复Bug，更是沉淀知识、升级AI能力的机会

★ 关键行动：建立"小步快跑 + 契约先行 + 测试护城河 + 持续进化"编码体系

测试与质量保障

AI时代测试不是变少了，而是变得更重要且更自动化

自动化测试 · 后置评审 · 五维度分类 · 测试护城河 · 质量度量

AI自动化测试越完备，程序调试速度越快

4h+

无测试 · 平均调试时间

覆盖率 0%

1h

单元测试30% · 平均调试时间

覆盖率 ~30%

20min

单元+集成60% · 平均调试时间

覆盖率 ~60%

5min

全覆盖95%+ · 平均调试时间

覆盖率 95%+

测试金字塔原则

少量E2E测试

中层集成测试

底层大量单元测试

负指数关系：测试覆盖率 ↑ → 调试时间 ↓↓

- ▶ 覆盖率每提升30%，调试时间缩短50%以上
- ▶ 无测试时AI定位Bug形同猜谜 —— 4小时+的反复试错
- ▶ 95%+覆盖率时AI可精确定位 —— 5分钟内定位根因

核心结论：测试越完备，AI定位Bug越精准 —— 测试是AI编码的加速器而非负担

一个Bug的三次后置评审：代码质量是如何逐轮收敛的

评审轮次收敛过程



后置四区评审法



核心洞察：一次评审不够 —— 将"评审→修复→再评审"作为迭代闭环，结构化方法论比经验更重要

五维度分类体系，实现测试全覆盖、零遗漏（一）

维度1 合法性区分



正向用例
验证正常功能

反向用例
验证异常处理

维度2 等价类覆盖



有效等价类
典型输入代表

无效等价类
非法输入代表

维度3 边界值校验



临界值测试
min/max/±1

极值测试
空/null/超长

维度4 范围分层



局部测试
单组件/单函数

跨功能+端到端
组件协作与全链路

维度5 风险猜错



极端场景
断网/超时

历史缺陷+并发
竞态条件

五大维度各司其职、互不交叉 —— 彻底解决用例设计凭经验的问题，实现全覆盖零遗漏

标准化用例模板

用例编号 TC-XXX-001

测试维度 合法性 / 等价类 / 边界值 / 范围分层 / 风险猜错

测试类型 正向 / 反向

前置条件 系统状态、数据准备

输入数据 具体输入值

用例设计顺序

- | | |
|------------|----------|
| 1. 合法性区分 → | 先区分正向/反向 |
| 2. 等价类覆盖 → | 有效+无效全覆盖 |
| 3. 边界值校验 → | 临界值+极值 |
| 4. 范围分层 → | 局部+全局 |
| 5. 风险猜错 → | 金字塔+异常场景 |

关键价值：新人也能设计出和老员工同等质量的测试用例 —— 让测试设计从艺术变为科学

✦ Q1 测试生成

从"人写测试" → "AI生成测试+人审核心逻辑"

AI依据五维度体系自动生成测试用例，人只需审查业务逻辑

效率提升：3-5倍

🕒 Q2 测试执行

从"定期执行" → "每次变更自动全量回归"

CI/CD触发自动化测试，每次代码提交都运行全量回归

频率提升：从每周到每次提交

🔄 Q3 测试维护

从"代码改了测试没改" → "AI同步维护测试与代码"

每次代码变更时AI自动更新相关测试用例，保持测试与代码一致

维护成本降低：80%

📊 Q4 测试分析

从"人工分析结果" → "AI自动分类+定位+建议修复"

AI自动分析失败原因、定位问题代码、推荐修复方案

分析速度：10倍提升

四个转变的本质：从"人驱动"到"AI驱动" —— 人聚焦策略和决策，AI负责执行和分析

第三层：质量门禁层

合并前必须通过所有层级检查，任何一项不通过自动拦截

不可跳过 · 自动拦截



第二层：CI层

每次提交触发全量回归测试 + 覆盖率检查 + 安全扫描，分钟级反馈

全量回归 · 覆盖率 · 安全扫描



第一层：本地层

开发者每次保存自动运行单元测试 + 坏味道扫描，秒级反馈

单元测试 · 坏味道扫描 · 秒级

关键原则：每一层都是自动化的、不可跳过的、AI辅助执行的 —— 构建真正的测试护城河


 $\geq 80\%$

测试覆盖率

应 $\geq$ 人类代码标准
越高的覆盖率=越快的调试


 < 0.5

缺陷密度

每千行代码缺陷数
对比AI与人类代码质量


 < 3.0

坏味道密度

每千行代码坏味道数
衡量代码整洁度的关键指标

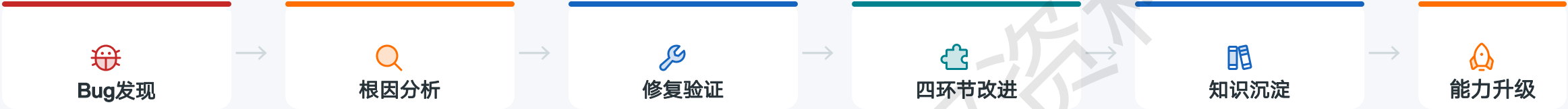

 $< 5\%$

变更失败率

变更导致回归失败概率
衡量系统稳定性的标尺

AI代码质量不能只看“能不能跑”，需要多维度度量体系

四项指标构成质量度量闭环：覆盖率看预防能力 · 缺陷密度看产出质量 · 坏味道密度看代码健康 · 变更失败率看系统韧性



知识沉淀的五步转化



核心价值：让每一次Bug都成为团队的永久资产 —— 不只是修复代码，更是沉淀知识和升级AI能力

✓	核心关系	测试完备度与AI编码效率正相关 —— 覆盖率每提升30%，调试时间缩短50%+
📁	方法论	五维度分类体系 + 后置评审迭代 + 需求探索与缺陷复盘统一
↻	策略转变	测试生成自动化、测试执行持续化、测试维护同步化、测试分析智能化
🛡️	护城河构建	本地 → CI → 质量门禁三层自动化防线，层层递进、不可跳过
📖	知识沉淀	每个Bug → 根因 → 教训 → 规范 → Skill → AI能力升级的完整闭环

核心理念

测试是AI编码的安全网 —— 测试越完备，AI编码越高效。在AI时代，测试不是变少了，而是变得更重要且更自动化。
关键行动：构建三层护城河，建立五维度分类体系，让每次Bug修复都成为知识资产

过程体系

将AI纳入软件过程管理，构建人机协作的过程体系

OSSP · PDP · Harness · CMMI · HMM成熟度 · LLM赋能 · 功能点度量

人类为什么爱敏捷

人类怕约束：繁琐的流程是人性天敌，人类天生抗拒标准化的束缚

人类怕文档：写文档、维护文档违背人之天性

人类爱灵活：敏捷强调"可工作的软件高于详尽的文档"

敏捷的本质：减少形式化，回归人与人的沟通协作

AI为什么爱规范

AI守规则：AI天生遵守规则，无惰性、不抵触、不偷懒

AI不怕文档：全流程留痕、标准化输出、量化度量正是AI的强项

AI不抵触流程：严格执行每一个CMMI过程域的所有要求

CMMI的本质：标准化、可度量、持续改进 —— AI的天然优势

核心洞察：真相不是CMMI过时，而是人性拒绝CMMI的繁琐 —— AI恰好能轻松落地CMMI的全流程

第三层：Harness（AI协同规则）

将过程规范转化为AI可识别、可执行、可校验的结构化规则 —— 四大组件：Principles / Rules / Skills / Specs

AI可执行 · 结构化 · 组织级-项目级-个人级三级分层



第二层：PDP（项目定制流程）

基于OSSP的项目柔性适配 —— 支持重装/轻装双模式，根据项目类型灵活调整过程资产

重装模式（全量过程） · 轻装模式（裁剪过程）



第一层：OSSP（组织标准流程）

企业通用研发基线 —— 定义组织级的软件工程标准，覆盖所有关键过程域

组织基线 · 通用标准 · 过程资产沉淀

Harness成熟度模型（HMM）：如何度量你的人机协作体系（一）

L1 初始级

个人英雄主义，无统一规范，AI使用全靠个人能力

L2 可管理级

项目级规则初现，团队开始共享Prompt和Skill，AI使用从个人走向团队

L3 已定义级

组织级标准确立 —— 统一的Skill库和知识库，标准化的AI协作流程

L4 量化管理级

数据驱动持续优化 —— 度量AI协作效率，量化质量指标，建立基线

L5 优化级

AI自驱动演进 —— AI主动识别改进机会，自动更新规则和Skill，持续自我优化

关键洞察

HMM衡量的是组织在AI协作方面的成熟度 —— 从L1的"个人英雄主义"到L5的"AI自驱动演进"，每一步都有明确的升级路径

每级关键能力

- L1→L2: 建立统一Prompt模板和Skill仓库
- L2→L3: 制定组织级AI协作标准和度量体系
- L3→L4: 数据采集和分析基础设施、量化基线建立
- L4→L5: AI自反馈闭环、自动规则优化
- L5 特征: AI主动发现改进机会 → 自动更新规则 → 持续优化

每一步升级都是可控的渐进式改进

自评问卷

- Q1: 团队是否有统一的AI使用规范？
- Q2: Skill是否跨项目复用？（至少3个项目）
- Q3: AI协作效率是否有数据度量？
- Q4: 代码生成质量是否有量化基线和趋势？
- Q5: 是否建立了AI自动反馈和改进闭环？
- Q6: 团队当前的HMM成熟度等级是？

核心协作模式

动态项目特征 + 静态组织资产 → 结构化提示词 → AI输出 → 人审核执行 → 经验沉淀

CMMI PLAN实践域 —— AI辅助结构化输出

🔗 WBS分解：AI根据需求自动生成工作分解结构

🕒 工作量估算：历史数据+功能点估算结合AI分析

📊 风险识别：基于历史项目数据自动标记风险

🔍 任务分配：基于技能矩阵自动匹配人员

📅 进度编排：依赖关系+资源约束自动排程

分工原则：人聚焦决策判断，AI负责信息处理

将LLM从聊天工具升级为CMMI过程的智能协作者 —— 让AI赋能过程执行，让人聚焦决策判断

PLAN 2.1

工作方法制定

动态知识→静态检索
→Prompt→AI→人审→沉淀

PLAN 2.2

技能规划

技能矩阵自动匹配
→AI推荐→人确认

PLAN 2.3

预算进度编制

历史数据+功能点估算
→AI生成排程

PLAN 2.4

干系人参与

自动识别干系人
→生成参与计划

PLAN 2.5

移交策划

AI生成移交清单
→人审核确认

PLAN 3.1

资源协调

资源依赖分析+冲突检测
→AI提出协调方案

PLAN 3.2

计划集成

多计划自动合并
→冲突检测→人决策

PLAN 4.1

量化过程设计

基线建立+指标定义
→AI辅助量化分析

每条实践遵循统一模式：动态知识输入 → 静态知识检索 → 结构化Prompt → AI执行 → 人审核 → 知识沉淀

CMMI关键过程域	→	Harness工程实践
RDM 需求开发管理	→	Spec规范 + 需求Skill —— 需求结构化、可追溯、可校验
PLAN+MC 计划与监控	→	Rules规则 + workflowSkill —— 计划生成、进度监控、偏差分析
CM 配置管理	→	版本控制Skill + 变更管理Rules —— 基线管理、变更追溯
PAD 过程资产开发	→	Skill库 + 知识库管理 —— 资产标准化、可复用、持续演进
VV 验证确认	→	测试Skill + 质量门禁Rules —— 自动化验证、不可跳过
MPM 绩效管理	→	HMM度量 + 量化分析Skill —— 成熟度评估、趋势分析
Harness工程解决"AI能不能把事做成", CMMI解决"AI是否持续可靠可控地把正确的事做成" —— 两者互补		

统一协作模式

动态项目特征+静态组织资产 → 结构化提示词 → AI输出 → 人审核执行 → 经验沉淀

RDM 需求开发管理 需求结构化·可追溯	TS 技术方案 AI辅助架构设计	PI 产品集成 自动化集成策略	PR 同行评审 AI辅助代码审查
VV 验证确认 测试自动化·全覆盖	PQA 质量保证 质量门禁·自动拦截	PLAN 计划 AI辅助排程估算	MC 监控 自动偏差分析
CM 配置管理 版本控制·变更追溯	PAD 过程资产 Skill·知识库管理	MPM 绩效管理 HMM度量·量化分析	... 共21个实践域 覆盖完整CMMI体系

核心理念：让AI赋能过程执行、让人聚焦决策判断、让数据驱动持续改进 —— 覆盖CMMI全部21个实践域



人机分工原则

人做决策：
判断 · 取舍 · 创新 · 决策

AI做执行：
生成 · 检查 · 记录 · 分析

分工越清晰，效率越高



标准先行原则

先有标准后有执行
AI需要明确的规则才能高效工作

模糊的标准 = 模糊的输出
标准化过程资产是AI协作的基础

无标准，不AI



持续度量原则

没有度量就没有改进
HMM五级提供清晰的升级路径

每个级别可量化衡量
L1→L2→L3→L4→L5 渐进升级

可度量，可改进，可进化

三大核心原则构成AI时代过程体系的基石：人机分工是前提，标准先行是基础，持续度量是保障

法则一 先想清楚，再让AI动手

想清楚一步，少做两遍功。方案先行，杜绝返工。

法则二 改哪里说哪里，圈外不动

改得越少，错得越少。冻结无关区域，精准修改。

法则三 先跑通主干，再完善枝叶

先能用，再好用。不做无用优化，逐步迭代。

真正的懒人智慧：用最省劲儿的方法做对事，避免无用功。不是啥都不做，而是先想清楚再做。

📄 需求与设计

- 1. 思路先行
- 2. 逆向评审
- 3. 方案对比
- 4. 任务拆解

懒在动工前

</> 编码实施

- 5. 圈定边界
- 6. 最小补丁
- 7. 伪代码翻译
- 8. 红绿蓝
- 9. 单一职责

懒在精准

↻ 迭代与优化

- 10. 主线优先
- 11. 渐进对话
- 12. 外科手术

懒在安全

🔧 调试与维护

- 13. 侦探破案
- 14. 代码考古
- 15. 边界测试

懒在高效

🛡️ 质量与工程化

- 16. 回归校验
- 17. 文档同步
- 18. 约束内化
- 19. 分层实现
- 20. 版本隔离

懒在持久

五大阶段覆盖完整开发流程：需求 → 编码 → 迭代 → 调试 → 工程化

每个阶段贯彻"懒人智慧"：用最少返工，做最对的事

💡 1. 思路先行 — 先出方案，再写代码

花5分钟确认方案，能省2小时返工。方案错了，写再多代码都白搭。

反面："帮我写个订单系统"→花一下午改bug

正面：先确认积分规则→一次生成就可

🔍 2. 逆向评审 — 先挑刺再动手

让AI找出方案里的坑（事务保护、并发控制、异常处理）

反面：转账无事务保护→A扣钱B没收，半夜对账

正面：AI指出4个坑→按建议完善→上线0故障

📄 3. 方案对比 — 你只负责选

让AI给多个方案（优缺点+适用场景），人按场景做选择

反面：直接生成令牌桶→小项目不需要那么复杂

正面：AI给出3种限流方案→选固定窗口→一次选对

🧩 4. 任务拆解 — 一次只做一件事

大需求→AI混乱→bug多逻辑乱。拆成子任务逐步实现。

反面：2000行代码混一个文件，根本没法改

正面：5个子任务逐步集成→逻辑工整直接能用

阶段核心：懒在动工前

确认方案 → AI挑刺 → 多方案对比 → 任务拆解 → 一次生成就可

关键转变：从"直接甩需求"到"先想清楚再让AI动手"

阶段二：编码实施 — 懒在精准，避免"擦屁股"

5. 圈定边界

明确"能改什么、不能改什么"

AI爱"顺手牵羊"重构代码：

改错误提示→改崩session

冻结区域+可改区域

6. 最小补丁

只输出修改片段，不输出整个文件

输出整个文件→一行行比对

500行文件只为找3行修改

一目了然，省时放心

7. 伪代码翻译

中文讲逻辑→AI翻译成代码

人懂业务不懂语法？AI当翻译

Rust代码看不懂不敢用→翻译

变量易懂+关键步骤注释

8. 红绿蓝（TDD式AI编程）

红：先写测试用例 → 绿：实现最小通过代码 → 蓝：优化重构

反面：直接生成折扣函数→改3轮才改对 | 正面：测试驱动→一次到位

9. 单一职责

一次只给AI一个任务，不混在一起说

反面：修bug+优化+加日志+整理格式→哪件都做不好

阶段核心：精准控制AI的修改范围，用测试驱动保证质量，一次只做一件事

10. 主线优先

先实现基础主线功能，跑通后再加细节。完美主义最费时间——一开始就想实现所有细节（断点续传+进度条+格式校验），结果连最基础的都跑不通。

分阶段交付，每阶段都有可用版本

11. 渐进对话

从简单到复杂逐步加需求。一口吃不成胖子，一次也写不出完美的复杂代码。
Step1→基础查询→Step2→分页→Step3→过滤→Step4→排序→Step5→缓存
某步出问题→快速定位，不排查全部

12. 外科手术

改核心代码前先评估影响范围。
id从int改string→订单、支付、日志模块全崩→通宵修关联代码。
先评估→制定迁移计划→逐步修改
平稳过渡，不连夜修bug

阶段核心：先主干后细节，逐步叠加需求，改前先评估——懒在安全

🔧 13. 侦探破案 — 懒得瞎改

遇bug先让AI诊断原因，再修复。
不知道bug原因就瞎改，改到天荒地老：
改状态机→不对→改查询逻辑→还是
不对。改了5轮还是没解决问题。

先诊断、再修复、一针见效

👁️ 14. 代码考古 — 懒得猜

先理解后修改，不理解就改=不知道
对不对。花5分钟理解，省1小时迷茫。
让AI：①比喻解释业务逻辑 ②指出
核心逻辑 vs 历史遗留 ③定位修改点

精准修改，不误伤核心逻辑

🛡️ 15. 边界测试 — 懒得被投诉

让AI主动找自己代码的漏洞。
日期解析函数只支持YYYY-MM-DD→
用户输入"2024-02-30"崩了。
让AI找3种崩溃输入→2月30日、

格式错误、空字符串→修复→健壮

阶段核心：先诊断再修复、先理解再修改、先找漏洞再上线——懒在高效

阶段五：质量与工程化 — 懒在持久，避免后续麻烦

☑ 16. 回归校验 — 懒得被坑

改完让AI自查：影响了哪些？原有功能是否被破坏？
加字段忘了更新序列化→改完输出回归报告

📄 17. 文档同步 — 懒得回忆

代码文档一起出：函数注释+使用示例+规则
三个月后看不懂自己的代码→顺手写文档

🔒 18. 约束内化 — 懒得记规则

规则写入代码断言，AI忘了代码也不会忘
assert order.total>0→规则固化，永远有效

📁 19. 分层实现 — 懒得拆垃圾

不分层代码是一坨，改一行翻半天。按层组织：router/service/repo
路由+业务+数据+模型全混一个文件→结构清晰，想改哪层改哪层

🔗 20. 版本隔离 — 懒得后悔

永不覆盖，保留历史。payment_v2.py→payment_v3.py→v2不变
优化性能改坏了→v2被覆盖找不回→改错了一键回滚到v2

阶段核心：回归校验→文档同步→约束内化→分层实现→版本隔离——懒在持久

20条工作模式速查表

阶段	模式	核心原则	一句话口诀
需求与设计	1. 思路先行	先出方案，再写代码	先想清楚一步，少做两遍功
	2. 逆向评审	让AI先挑刺，再动手	先找坑再写代码，省得踩坑
	3. 方案对比	AI给方案，人做选择	把纠结的事交给AI
	4. 任务拆解	大需求拆成小任务	一次只做一件事
编码实施	5. 圈定边界	能改什么、不能改什么	冻结无关区域，精准修改
	6. 最小补丁	只输出修改片段	不比对500行就找3行
	7. 伪代码翻译	中文逻辑→AI翻译代码	不懂语法也能写出代码
	8. 红绿蓝	先测试→实现→优化	"必须通过测试"
	9. 单一职责	一次只做一件事	多任务混杂→一团糟
迭代与优化	10. 主线优先	先主干后细节	先能用，再好用
	11. 渐进对话	逐步叠加需求	一口吃不成胖子
	12. 外科手术	先评估影响再动手	平稳过渡，不连夜修bug
调试与维护	13. 侦探破案	先诊断bug原因再修复	先诊断、再修复、一针见效
	14. 代码考古	先理解后修改	不理解就改=不知道对不对
	15. 边界测试	让AI主动找漏洞	AI自己找茬比被用户发现强
质量与工程化	16. 回归校验	改完必自查	别等"怎么这里也坏了"
	17. 文档同步	代码文档一起产出	三个月后看不懂→顺手写
	18. 约束内化	规则写入代码断言	AI忘了代码也不会忘
	19. 分层实现	按层组织代码	想改哪层改哪层
	20. 版本隔离	永不覆盖，保留历史	一键回滚比后悔药管用

真正的"懒人"用最省劲儿的方法做对事。想清楚→少返工，圈边界→少出错，分步来→少焦虑。

🧠 人机分工清晰

人：出思路、定方向、做判断
AI：枚举方案、生成代码、诊断问题
各司其职，效率最大化

🎯 精确控制边界

告诉AI改什么、不改什么
任务拆解、圈定范围、最小修改
改得越少，错得越少

🚀 小步迭代验证

先跑通主干，再完善枝叶
逐步加需求，每步都确认
一次成型，不用返工

懒人AI结对编程完整 workflow

确认方案 (思路先行) → 风险排查 (逆向评审) → 精准编码 (圈定边界) → 测试验证 (红绿蓝) → 迭代优化+版本隔离

回到三大懒人法则

法则一：先想清楚，再让AI动手

法则二：改哪里说哪里，圈外不动

法则三：先跑通主干，再完善枝叶

	核心洞察	人类爱敏捷，AI爱CMMI —— 两者互补而非对立，构成人机协作的黄金搭档
	体系架构	OSSP基线 → PDP适配 → Harness执行，三级递进覆盖组织-项目-个人
	成熟度模型	HMM五级模型，从L1个人英雄主义到L5 AI自驱动演进，逐步升级
	LLM赋能CMMI	覆盖全部20个实践域的AI协作方案，统一的"动态知识+静态资产→AI→人审"模式
	关键行动	建立Harness体系，推动过程成熟度持续升级，将AI纳入过程管理的每个环节

核心理念

AI时代的过程体系不是抛弃传统工程方法，而是用AI重新激活它们 —— CMMI不再是负担，而是在AI加持下成为效率倍增器。

关键行动：评估当前HMM等级 → 识别短板 → 制定升级计划 → 建立Harness → 持续度量改进

01 认知重塑
思想基础

02 工程哲学
价值观与方法论

03 需求与设计
输入质量

04 编码
执行效率

05 测试与质量保障
安全网

06 过程体系
组织能力

核心公式

AI4SE成功 = 正确认知 × 工程哲学 × 精准需求 × 规范编码 × 完备测试 × 成熟过程 × 持续落地

一个核心理念

AI不是银弹，但是最强辅助——拥抱AI效率红利，清醒认知能力边界，回归工程本质



AI负责确定性

人类负责探索不确定性



去模糊化 + Skill体系

AI协同的两大基础设施



持续迭代 持续进化

让每次变更都成为团队资产

谢谢

欢迎提问交流

AI4SE —— 人工智能时代的软件工程思想与实践
从认知重塑到能力落地，构建AI时代软件工程新范式